FIXFLOW

Project Overview & Technical Guide

A detailed guide to the maintenance-management application in this repository: what it does, how its components fit together, how a repair travels through the system, and how to run it locally.

At a glance	Implementation
Frontend	React 18, Vite, React Router, Axios, Tailwind CSS
Backend	Python, FastAPI, Pydantic, SQLAlchemy
Database	PostgreSQL 16 (Docker Compose)
Predictive analytics	Pandas, NumPy, scikit-learn; XGBoost when available
Local addresses	UI: http://localhost:5173 | API: http://localhost:8000 | API docs: /docs

Scope note: This document describes the repository as inspected on 23 October 2026 and the local setup changes made while resolving startup errors. Code and Git history do not establish which lines were authored by Claude or another AI tool.

Prepared for learning and project understanding ? Local development edition

Contents

1. What FixFlow is
2. Architecture and request flow
3. Roles and user journeys
4. Maintenance request lifecycle
5. Database and domain model
6. Authentication and access control
7. SLA, notifications, audit, and scheduled work
8. Failure prediction and preventive maintenance
9. API and frontend map
10. Running the project locally
11. Troubleshooting history
12. Limitations, safety, and next steps
13. How to explore the code

1. What FixFlow is

FixFlow digitizes the operational lifecycle of facility maintenance. Instead of tracking faults through messages or spreadsheets, it gives a reporter a structured way to identify an asset and describe a problem, then gives technicians and facility staff a shared request record.

The main functional loop is: report a fault ? set urgency and an SLA deadline ? find a suitable technician ? record work and parts ? resolve and rate ? use the history to improve oversight and estimate future risk.

The system combines day-to-day operations (requests, assignments, status, costs, notifications) with administrative capabilities (users, service policies, reports, backups) and an experimental predictive-maintenance feature.

What it is not

- It is not a hosted service or a cloud deployment: the provided commands run the app locally on the developer computer.
- The risk score is an estimate based on available history, not a guarantee that an asset will fail.
- The current root README contains only the project title; implementation details were therefore verified from source files rather than inferred from README documentation.

2. Architecture and request flow

There are three primary runtime components. The browser downloads the React app from Vite. The app sends JSON, form-data, and file-download requests to FastAPI. FastAPI authenticates and validates requests, runs business rules, then reads or writes PostgreSQL through SQLAlchemy.

Layer	Technology / responsibility
Browser UI	React pages and components. React Router selects pages; AuthContext stores the signed-in user state.
Dev web server	Vite serves the frontend at port 5173 and produces optimized static assets for a production build.
HTTP client	Axios uses API base URL http://localhost:8000 by default, attaches the bearer token, and maps common API errors.
API	FastAPI routers implement endpoints; Pydantic schemas validate inputs; dependencies provide DB sessions and identity/role checks.
Persistence	SQLAlchemy ORM models map Python classes to PostgreSQL tables. PostgreSQL is provided by the postgres:16 Docker image.
Background work	APScheduler runs within the API process for SLA checks, workload correction, predictions, and retraining.

Data path for a browser API call

```
Browser page ? frontend/src/lib/api.js ? HTTP request with Authorization: Bearer <JWT>
? backend/app/main.py and a router ? dependency opens SQLAlchemy Session
? business logic reads/writes PostgreSQL ? JSON response ? React updates UI
```

The API enables CORS for the local frontend origins (localhost and 127.0.0.1 on port 5173). Uploaded files are served by the API from its uploads directory. The API health route is /api/health; Swagger documentation is at /docs.

Important entry points

File	Why it matters
frontend/src/main.jsx	Mounts the React app and providers into the browser document.
frontend/src/App.jsx	Declares routes and client-side role-based page restrictions.
frontend/src/lib/api.js	Central Axios client, API origin, token header, and error-message helper.
backend/app/main.py	Creates FastAPI, installs middleware/handlers, includes routers, mounts uploads, starts scheduler.
backend/app/config.py	Loads settings from backend/.env and creates runtime storage directories.
backend/app/database.py	Creates SQLAlchemy engine, Base, session factory, and get_db dependency.
backend/app/models.py	Defines tables, constraints, constants, and ORM relationships.

3. Roles and user journeys

Role	What they do	Main UI
Reporter	Create requests, view their own requests, upload photos, review resolution, leave feedback, reopen if needed.	My Requests; Report a Problem; Request Details
Technician	See assigned jobs, accept/start/pause work, document repair and parts, complete the work, update availability and skills.	My Jobs; Technician Profile
Manager	See operational overview, manage requests/assignments, monitor SLAs and technicians, maintain assets, inspect risk, export reports.	Manager Overview; Requests; SLA; Technicians; Assets; Risk; Reports
Admin	Manage users and catalog/settings, inspect logs/model status, train prediction model, create/restore backups.	Users; Catalog; Settings; Model Center; Logs; Backups

The UI routes are centralized in frontend/src/App.jsx. Unauthenticated users are sent to sign-in; signed-in users are sent to a role-specific home page. Attempting to open a restricted route redirects a user to their permitted home.

This client-side routing improves usability, but it is not the security boundary: backend endpoints independently check identity and role. For example, user administration and model training are restricted by backend dependencies.

Registration versus demo accounts

Self-registration creates a Reporter role. Staff roles are assigned by administrators or provisioned by the demo seed script. The login page shows demo account shortcuts; these only work if the matching demo users exist in the database.

4. Maintenance request lifecycle

Each request represents a reported problem for one asset. It stores who reported it, who is assigned, severity, status, timestamps, due date, photos, repair logs, SLA policy, feedback, escalations, and audit history.

Status	Meaning
Open	Reported; waiting for a suitable assignment or staff action.
Assigned	Assigned to a technician, who has not necessarily accepted yet.

Status	Meaning
In Progress	Technician has started work; work timer can be running.
On Hold	Technician paused the work; the request remains active for workload accounting.
Resolved	Technician completed repair. Reporter can confirm/rate or reopen.
Closed	Work has been accepted/closed, often after feedback.
Reopened	Previously resolved/closed work was reported as still faulty; SLA window restarts.

Typical reporter flow

- Choose the asset by building/floor/room, or scan its QR label. A QR code opens the report form with the asset preselected.
- Provide a short title, details, severity, and optional before-repair photos.
- On submission, the backend sets a due time from the relevant SLA policy, marks the asset faulty when appropriate, attempts technician assignment, records audit activity, and creates notifications.

Typical technician flow

- Review jobs ordered by urgency/deadline, accept the assignment, then start work. Starting records the acceptance if needed and begins the work timer.
- Pause stops the timer; resuming starts it again. Repair logs can record interim notes, parts, labour, and cost.
- Completing the job stops the timer, records a repair log and resolution time, updates workload/asset state, and notifies the reporter.

Assignment rule

The assignment service maps an asset category to a required skill, filters to active available technicians with the matching skill, and chooses the technician with the lowest current workload (ties use technician ID). If no candidate exists, the request remains unassigned and managers are notified. A manager can manually assign or reassign.

State and consistency

The request endpoints validate allowed transitions. Workload counts include Assigned, In Progress, and On Hold states. Reassignment releases the former workload slot and adds the new one. A periodic recount recalculates workload from active assignments as a safety net.

5. Database and domain model

SQLAlchemy maps domain classes to PostgreSQL. Foreign keys express ownership/relationships; unique, check, and not-null constraints enforce important invariants in the database as well as the API. The users table is named users because user is a PostgreSQL reserved word.

Table / group	Purpose
users	Identity, email, password hash, role, active state, profile.
locations	Building, floor, room; locations can contain multiple assets.
assets	Asset identity, category, location, QR code, purchase/warranty dates, status.

Table / group	Purpose
technicians, skills, technician_skills	Technician profile/workload and many-to-many skill qualifications.
asset_categories	Admin-managed category-to-required-skill mapping used by assignment.
sla_policies	Response/resolution targets by severity.
maintenance_requests	Core fault ticket and lifecycle timestamps, assignment, SLA, due date, work timer, reopen count.
request_photos	Before/after evidence associated with a request.
repair_logs	Work notes, parts, labour, repair cost, downtime, technician and completion timestamp.
escalations	SLA warning/breach/response escalation records and acknowledgement state.
feedback	One rating/comment per request.
failure_predictions	Timestamped per-asset scores, risk bands, and model version.
notifications	In-app notifications and read state.
audit_logs	Actor/action/entity/details timeline for traceability.
system_settings	Persisted configurable feature settings.
model_runs	Training metadata, metrics, feature importance, active model run.

Relationships and data notes

- One location can contain many assets; one asset can have many requests and predictions.
- A request belongs to an asset and reporter; it may be assigned to a technician and SLA policy.
- Repair logs and photos are children of requests. Feedback is one-to-one with a request.
- Technician-to-skill is many-to-many through technician_skills.
- Some fields, such as request work-timer timestamps and preventive-maintenance flags, are explicit extensions supporting features beyond the basic entities.

The table definitions are in backend/app/models.py. The project currently initializes tables via SQLAlchemy create_all in app.init_db; the alembic versions folder has no migration scripts, so a versioned schema-upgrade history is not currently available.

6. Authentication and access control

On login, the backend looks up the email and verifies the supplied password against a bcrypt hash. It returns a signed JWT with a subject user ID, role, issue time, and expiration. Passwords are not stored as plaintext. Registration follows the same hashing path.

The frontend stores the access token in localStorage and Axios attaches it as a bearer token on API calls. On an expired/invalid session (401), the client clears the token and redirects to the login page. AuthContext obtains the current profile from /api/auth/me when a saved token exists.

FastAPI dependencies validate the JWT, fetch the user from the database, reject inactive users, and enforce allowed roles. Routes should always be protected server-side; hiding a link in React alone does not prevent access to an API endpoint.

Security note: The checked-in local environment is for development. Before any shared or production deployment, use unique strong secrets and passwords, keep .env private, restrict database and admin access, configure trusted origins, and review file upload and backup behavior. Do not publish local credentials.

7. SLA, notifications, audit, and scheduled work

Service-level targets

Severity	Response target	Resolution target
Critical	1 hour	8 hours
High	4 hours	24 hours
Medium	8 hours	72 hours
Low	24 hours	168 hours

SLA calculations are centralized in backend/app/services/sla.py. A request can be on_track, at_risk, or breached while unresolved, and met or missed after resolution. Reopening starts a new resolution window based on the policy.

Escalation behavior

The scheduler checks unresolved requests that have due dates. It can create response-time escalations when no one has accepted in time, warnings after a configured percentage of the window, and breaches after the resolution deadline. It avoids duplicating a given escalation level within the active SLA window. In-app notifications are sent to configured manager/admin audiences and, for resolution breaches, the technician.

Notifications and audit trail

Notifications are database-backed and visible in-app. Email is optional; with SMTP disabled, notification email content is logged rather than sent. Audit rows record important actions?such as request creation, assignment, status changes, login, exports, model training, and backup operations?and are used to build timelines and administer the system.

Jobs that run automatically

Job	Schedule	Purpose
SLA checks	Every configured interval (default 5 minutes)	Escalate response delays, at-risk requests, and resolution breaches.
Predictions	Nightly at 02:00 UTC	Score active assets and notify managers about newly high-risk assets.
Model retraining	Sunday at 03:00 UTC	Retrain if enabled and enough history exists.
Workload recount	Hourly	Correct technician workload counters from actual active assignments.

These jobs run inside the API process by default. If multiple API worker processes are started, only one should run the scheduler; the code comments recommend disabling it on extra workers or using a separate scheduler process.

8. Failure prediction and preventive maintenance

The goal is to estimate whether an asset will have a non-preventive failure report in the next 30 days. This makes the feature a prioritization aid for inspections, not a deterministic maintenance calendar.

Training data design

- The engine takes historical snapshots at 14-day steps. For each snapshot date, it computes features using only information available before that date.
- The label is whether a failure is reported for that asset in the following 30 days. Preventive requests are excluded from failure history.
- The latest 25 percent of snapshot dates are used for time-based evaluation when possible; a stratified random split is the fallback if the time split lacks both classes.
- After evaluation, the model is refit on all available labelled data and saved as a joblib bundle. Training-run metadata is stored in model_runs.

Features and algorithms

Feature family	Signals
Asset age / warranty	Age in days and whether warranty remains active.
Failure history	Total historical failures, failures in the last 90 days, time since last failure.
Repair history	Time since last repair, average repair cost, cumulative downtime.
Category	One-hot encoded asset category.

XGBoost is selected if the package imports successfully; otherwise the engine uses a scikit-learn Random Forest. If there is insufficient history to train, a transparent logistic-style heuristic score is used so fresh installations can still display risk levels. The default risk cutoffs come from system settings.

The risk API can return the latest score per asset, feature values, plain-language contributing reasons, whether preventive work is already scheduled, and a recommendation. Staff can schedule a preventive request for a future date; it then follows the ordinary assignment and SLA workflow.

Metrics such as ROC AUC, accuracy, precision, and recall are stored for model monitoring. Real usefulness depends on having enough representative, accurately logged repair history; synthetic demo records are useful for exploring screens but are not evidence of operational predictive accuracy.

9. API and frontend map

Backend router groups

Prefix / group	Main responsibilities
/api/auth	Register, login, OAuth2 token, current profile and password.
/api/requests	Create/list/view requests, photos, technician actions, repair logs, feedback, assignment and transitions.
/api/assets; /api/locations	Asset catalog, QR lookup/generation, location management, history.
/api/technicians; /api/users	Technician availability/profile/workload and admin user management.
/api/sla; /api/notifications	Policies, SLA monitor/escalations, read state.
/api/analytics; /api/reports	Operational summaries/trends and CSV/PDF exports.
/api/predictions	Score, train, inspect model/risk, schedule preventive work.
/api/admin; /api/skills; /api/categories	Settings, audit/system logs, usage, backups, skill and category catalog.

Open http://localhost:8000/docs to browse the generated OpenAPI docs, inspect request/response schemas, and try authorized endpoints. The /api/health response includes a database status check.

Frontend page families

- Reporter: reporter/MyRequests.jsx, reporter/NewRequest.jsx, RequestDetail.jsx.
- Technician: technician/TaskQueue.jsx, technician/TechProfile.jsx.
- Manager: manager/Overview.jsx, AllRequests.jsx, SlaCenter.jsx, Technicians.jsx, Assets.jsx, RiskScores.jsx, Reports.jsx.
- Admin: admin/Users.jsx, Catalog.jsx, Settings.jsx, ModelCenter.jsx, Logs.jsx, Backups.jsx.
- Shared layout, table, QR scanner, and UI primitives live under frontend/src/components; date formatting and API hooks are in frontend/src/lib.

10. Running the project locally

Use three terminals and leave each process running. The commands below assume Docker Desktop and Node.js are installed. The backend virtual environment and packages were already installed during the current setup; repeat the install command only if setting up a fresh checkout.

Terminal 1: database (run from repository root)

```
cd "C:\MCA\Sem III (Mini Project)\FixFlow\fixflow"
docker compose up -d
docker compose ps
```

The database service is PostgreSQL 16. Current local configuration maps host port 5433 to container port 5432 because a separate Windows PostgreSQL process was already listening on host port 5432.

Terminal 2: backend

```
cd "C:\MCA\Sem III (Mini Project)\FixFlow\fixflow\backend"
.\.venv\Scripts\Activate.ps1
python -m app.init_db
python -m uvicorn app.main:app --reload
```

If PowerShell blocks activation in this terminal, run: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, then activate again. On a fresh setup, create the environment and install dependencies first: py -m venv .venv, then .\.venv\Scripts\Activate.ps1 and python -m pip install -r requirements.txt.

Terminal 3: frontend

```
cd "C:\MCA\Sem III (Mini Project)\FixFlow\fixflow\frontend"
npm install
npm run dev
```

Open the Vite Local address, usually http://localhost:5173/. Check http://localhost:8000/api/health for API/database health and http://localhost:8000/docs for API docs.

Initialize versus seed

python -m app.init_db creates missing tables and ensures default SLA policies, skills, categories, and an initial Admin. python -m app.seed adds a sizeable college-campus demo dataset and demo users (Reporter student@fixflow.dev / Reporter@123; Technician ravi@fixflow.dev / Tech@123; Manager manager@fixflow.dev / Manager@123; Admin admin@fixflow.dev / Admin@123).

The seed script refuses to add demo requests if requests already exist. Do not use python -m app.seed --reset unless you deliberately intend to drop and recreate all tables and erase the existing data.

11. Troubleshooting history

Symptom	Cause	Resolution / current status
npm ENOENT for root package.json	npm was run from repository root; package.json belongs to frontend/.	Run npm commands from frontend. The production Vite build succeeded there; it emitted a large-bundle warning but completed.
PostgreSQL password authentication failed on 127.0.0.1:5432	A separate Windows PostgreSQL process owned host port 5432, so the backend reached the wrong server instead of the Docker database.	Docker Compose mapping changed to host 5433 ? container 5432, backend/.env points to 127.0.0.1:5433, and Docker role password was synchronized. Direct SQLAlchemy access and init_db succeeded.
Health request returned degraded	The already-running backend had loaded the earlier DB URL/password before config changed.	Restart the backend so it reloads backend/.env. Later /api/health returned status=ok and database=true.
GET /favicon.ico 404	The browser requested a favicon from the backend while the screenshot showed API requests.	Harmless to core functionality. The frontend index.html defines an inline SVG favicon; Vite serves the frontend icon when loading the UI.
Demo Reporter login rejected	Visible demo credentials are shortcuts, not proof that demo users already exist.	Run python -m app.seed once if the database has no requests; use student@fixflow.dev / Reporter@123. Do not reset existing data.

Last verified during troubleshooting: Docker database healthy on port 5433, database initialization succeeded, direct backend database access succeeded, and the API health endpoint returned {"status":"ok","database":true}. A running backend may need restarting after a .env edit.

12. Limitations, safety, and next steps

Observed gaps

- There is no substantive README documentation yet; this guide is intended to fill the explanatory gap, not replace developer docs.
- No Alembic migration scripts were present in backend/alembic/versions. create_all is suitable for bootstrapping but does not provide a controlled schema evolution strategy.
- No dedicated project test suite was found in the source inventory. A successful frontend build, Python syntax check, and live health check cover only limited aspects of correctness.
- The Vite build warns that the main JavaScript chunk exceeds 500 kB after minification. The build still succeeds; code splitting could improve initial load performance.
- Email is disabled by default. Demo data is synthetic. Model quality requires real, representative, consistently recorded history.

Operational cautions

- Keep backend/.env private and never put its secrets into screenshots, source control, or public issue reports. Change development credentials before any non-local use.
- A backup restore replaces current data. Reset seeding drops all tables. Neither action should be run casually.
- Uploaded files, database ports, JWT lifetime and CORS settings need production-specific hardening before deployment.

Reasonable next engineering steps

- Write a README covering prerequisites, setup, environment configuration, demo credentials, and common errors.
- Add Alembic migrations and tests for authentication, authorization, request transitions, assignment, SLA deadlines, and ML feature construction.
- Add frontend route/component tests and an end-to-end test for report ? assign ? resolve ? feedback.
- Introduce structured deployment config, secret management, controlled file storage, and a production deployment checklist.
- Review ML evaluation over time and compare predictions to observed outcomes before using risk tiers as operational policy.

13. How to explore the code

Question	Start here
Where do pages and role routing live?	frontend/src/App.jsx, then frontend/src/pages/ by role.
How do API calls work?	frontend/src/lib/api.js and backend/app/routers/.
Where is login/security?	backend/app/routers/auth.py, backend/app/security.py, frontend/src/lib/auth.jsx.
What is stored in PostgreSQL?	backend/app/models.py and backend/app/database.py.
How does assignment work?	backend/app/services/assignment.py and request creation/assignment endpoints.
How are deadlines computed?	backend/app/services/sla.py and backend/app/scheduler.py.
Where is risk prediction?	backend/app/ml_engine.py and backend/app/routers/predictions.py.
How are demo records created?	backend/app/init_db.py for reference data; backend/app/seed.py for demo data.
Where is configuration?	backend/app/config.py and the private local backend/.env.
How do I inspect the API?	Run backend, then open http://localhost:8000/docs.

A useful reading order is: main.py ? config.py/database.py ? models.py ? security.py ? one router (requests.py) ? the service it calls (assignment.py or sla.py) ? matching frontend page ? API client. Then inspect scheduler.py and ml_engine.py for the background and analytics features.

End of overview. This guide describes the inspected repository state and local setup; verify configuration and health endpoints if the environment has changed since 23 October 2026.